

P2P Trading Bot

Jacob Huynh, Henry Chen, Haoxuan Luo

Problem: Detecting Mispriced NBA Prediction Markets

Kalshi markets:

- **Yes/No contracts on real-world outcomes**
- **Prices reflect market-implied probabilities**

Our focus:

- **Team winner markets**
- **Player prop markets: points / rebounds / assists line**

Challenge:

Market prices move quickly and depend on many signals: team strength, recent player performance, injuries, lineup changes, news, and live market behavior.

Goal:

Use Agentic AI to identify potentially mispriced contracts for paper trading.

Why This Problem Matters

Prediction market prices aggregate beliefs from many participants, but they are not always perfectly efficient.

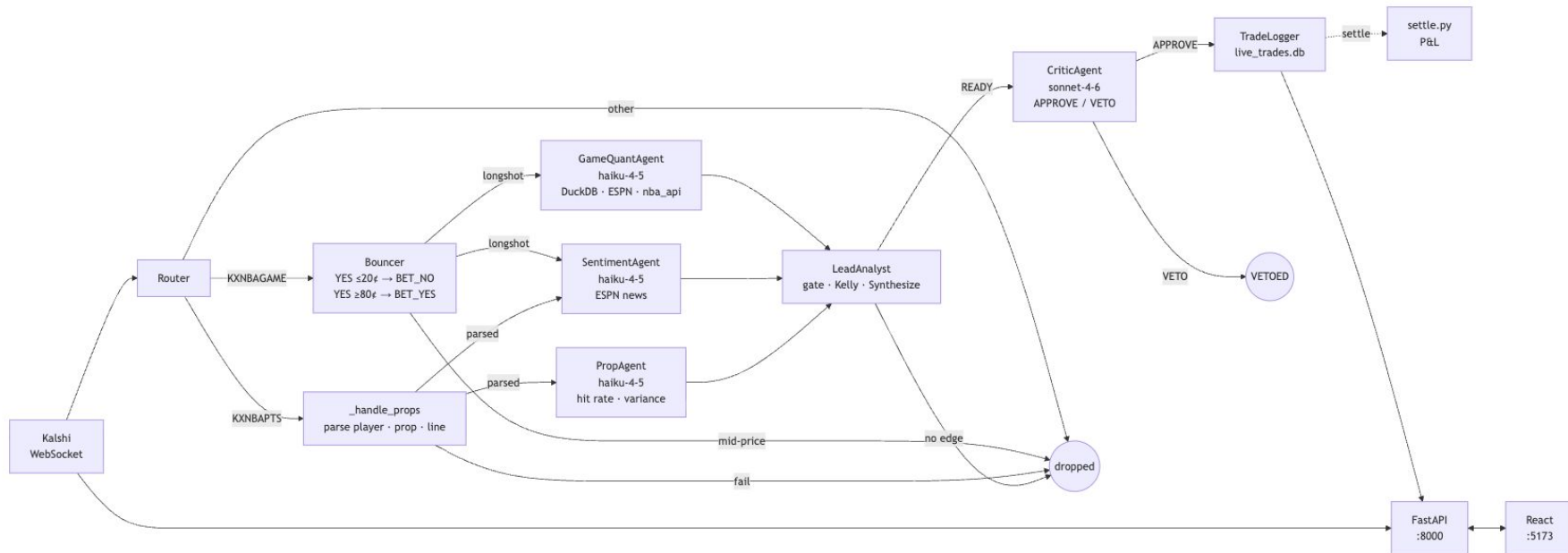
Mispricing can come from:

- **Public bias toward popular teams / star players**
- **Overreaction to recent games**
- **Slow reaction to injuries and lineup changes**
- **Fragmented information: Market data, player statistics, live news**

Why Agentic AI:

- **Requires using tools and collecting data**
- **Combines quantitative analysis with live context**
- **Needs coordination between specialized agents**
- **Requires a critic/risk-control step before making decisions**

System Design



Key Techniques – Parallel Agents

GameQuantAgent (haiku-4-5)

- Queries historical Kalshi parquet (DuckDB) for win rate vs. implied probability at this price
- Enriches with ESPN live score + nba_api team records and top scorers
- Verdicts: EDGE_CONFIRMED (gap > 2%, $n \geq 200$), EDGE_WEAK, NO_EDGE, INSUFFICIENT_DATA

PropAgent (haiku-4-5)

- Pulls player's last-10 game log from nba_api, computes hit rate vs. the prop line
- Also fetches usage rate and matchup history vs. opponent
- Verdicts: EDGE_CONFIRMED (hit > 65%) · EDGE_WEAK (55–65%) · NO_EDGE · INSUFFICIENT_DATA

SentimentAgent (haiku-4-5)

- Fetches ESPN news, filters to the relevant teams or player, LLM writes a 2–4 sentence summary
- Output: sentiment_context string injected into the trade packet (or None if unavailable)

Key Techniques – Decision Agents

LeadAnalyst (haiku-4-5)

- Waits for parallel agents, then gates in Python
 - PASS immediately if no edge or insufficient data (no LLM wasted)
- Synthesizes quant/player prop + sentiment into a single narrative paragraph for the Critic
- Computes Kelly fraction in Python (15% cap for game winners, 5% for props)

CriticAgent (sonnet-4-6)

- Sole job: find reasons to VETO
- First checks hard rules in Python (opposing same-game position, duplicate bet, \$15 exposure cap) – blocks before any LLM call
- LLM adversarially hunts: data contamination, overfitting, Kelly overbetting, liquidity, portfolio concentration, prop-specific flags (hit_rate > 90%, high variance, injury news)

Key Techniques – Tools

DuckDB + Parquet

- Local historical Kalshi NBA trade data, queried by price bucket across all finalized markets
- Gives GameQuantAgent the calibration gap and sample size

ESPN API

- Live scoreboard (score, game status, winner) and NBA news feed
- Used by GameQuantAgent for live game context and SentimentAgent for news filtering

NBA API

- Team last-10 W/L records and top-3 scorers per team (GameQuantAgent)
- Player game logs, usage rate, and matchup history vs. opponent (PropAgent)

Kalshi REST

- Fetches market title and rules so the router can parse player name, prop type, and line
- Also used by settle.py to poll for final results and compute P&L

Demo

<https://youtu.be/SjMW74vRIE4>

Results

TRADES EVALUATED

90

WIN RATE

98.9%

89/90

TOTAL P&L

+\$125.20

ROI

+14.0%

staked \$897.80

P&L by confidence

aggregated by orchestrator confidence



Contributions

Henry Chen	Frontend, Backend, Sentiment Analysis Agent, Agent Tools, Trades Evaluation
Jacob Huynh	Orchestrator Agent, Critic Agent, Quant Agent, Database Management, Kalshi Integration
Haoxuan Luo	Trade Logging, Trades Resolution

Codebase PR

<https://github.com/Qdata4Capstone/uva-26-spring-gen-agent-students-projectA/pull/16>